

EECS 482

Introduction to operating systems

Lock implementation;
deadlock

Peter Chen
University of Michigan

Lock implementation #2

(uniprocessor, no busy waiting)

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        add thread to queue of  
            threads waiting for lock  
        enable interrupts  
        switch to next ready thread  
        disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
            ready queue  
    }  
    enable interrupts  
}
```

Lock implementation #3 (uniprocessor, no busy waiting, handoff)

```
lock() {
    disable interrupts
if while (status != FREE) {

        add thread to queue of
            threads waiting for lock

        switch to next ready thread
    } else {
        status = BUSY
    }
    enable interrupts
}
```

```
unlock() {
    disable interrupts
    status = FREE
    if (any thread is waiting
        for this lock) {
        move waiting thread to
            ready queue
        status = BUSY
    }
    enable interrupts
}
```

How to provide atomicity for OS code?

- Remember: can't use mutex or semaphores
- What breaks atomicity for a section of code?
 - ~~Code may call yield, etc.~~ Don't do this ☺
 - ~~An interrupt may occur~~ Disable interrupts around critical sections. Use switch to “handoff” responsibility for interrupts to next thread.
 - Another processor may execute instructions

Atomicity on multiprocessor

- Disabling interrupts prevents current thread from being switched out on that processor, but threads may be running on **other** processors
- Could use atomic load and atomic store
 - Example: “Too much milk” solution #3
- Modern processors make it easier with instructions that atomically {read and write} memory

test_and_set

- Atomically write 1 to a memory location and return **old** value

```
test_and_set (X) {  
    old = X;  
    X = 1;  
    return old;  
}
```

atomic

- In Project 2, `std::atomic<bool>` provides similar operations

Lock implementation #4

(multiprocessor, busy waiting)

```
lock () {  
    while (test_and_set(status) == 1) {}  
}
```

atomic

```
unlock () {  
    status = 0  
}
```

atomic

- status=0: lock is free
- status=1: lock is busy
- Uses busy waiting
- This is called a spin lock

Busy waiting

- Some busy waiting is unavoidable on multiprocessors
- With lock implementation #4, how long could lock() busy wait?

```
lock()  
if (no milk) {  
    buy milk  
}  
unlock()
```

- How to minimize busy waiting?

Lock implementation #5

(multiprocessor, minimal busy waiting)

```
lock() {  
    disable interrupts  
    while (test_and_set(guard)) {}  
    if (status != FREE) {  
        add thread to queue of  
        threads waiting for lock  
  
        switch to next ready thread  
    } else {  
        status = BUSY  
    }  
    guard = 0  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    while (test_and_set(guard)) {}  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
        ready queue  
        status = BUSY  
    }  
  
    guard = 0  
    enable interrupts  
}
```

How long could lock() keep interrupts disabled?
How long could lock() busy wait on guard?

Switch invariant

Multiprocessor

- ~~Uniprocessor~~ switch invariant
 - When calling switch: interrupts must be disabled and guard must be 1
 - When switch returns: thread may assume interrupts are “still” disabled and guard is “still” 1
 - Remember: before running user code, interrupts must be enabled and guard must have been set to 0

Summary of lock solution

- Use low-level mechanisms to provide mutual exclusion for OS code
 - Disable interrupts
 - Spin lock (for multiprocessors)
- OS provides higher-level mechanism (locks) to provide mutual exclusion for user-level code
 - User code runs with interrupts enabled
 - User code runs with no spin locks held (by that CPU)

Summary of lock solution

- Hard problem: how to atomically add thread to a waiting list and sleep
- How did we achieve this?
 - If there is another thread to run: don't sleep! Instead, switch to other thread (obeying switch invariant)
- But what if there is no other thread to run?
 - Need hardware support to atomically enable interrupts and suspend CPU
 - HLT (x86), WFE (ARM), interrupt_enable_suspend (Project 2)

Project 2

- Write and test the project incrementally
 - 1 CPU, no interrupts
 - cpu and thread constructor
 - yield
 - mutex, cv, join
 - Add support for interrupts
 - Add support for multiple CPUs

Project 2

- Use lots of assertions
 - Assert any property that you expect to be true
 - Helps catch errors closer to where they occur
- Example: switch to null thread context. Would you rather:
 - Stop when you switch to the bad thread?
 - Stop when you put the bad thread onto the ready queue?
 - Stop when you put the bad thread onto a waiting queue?
- Example: At any time, a thread context must only be in one “place”
 - Where can a thread be?

Deadlock

Interviewer to EECS 482 student: “*Explain deadlock to me, then we’ll give you the job*”.

EECS 482 student to interviewer: “*Give me the job, then I’ll explain deadlock to you*”.

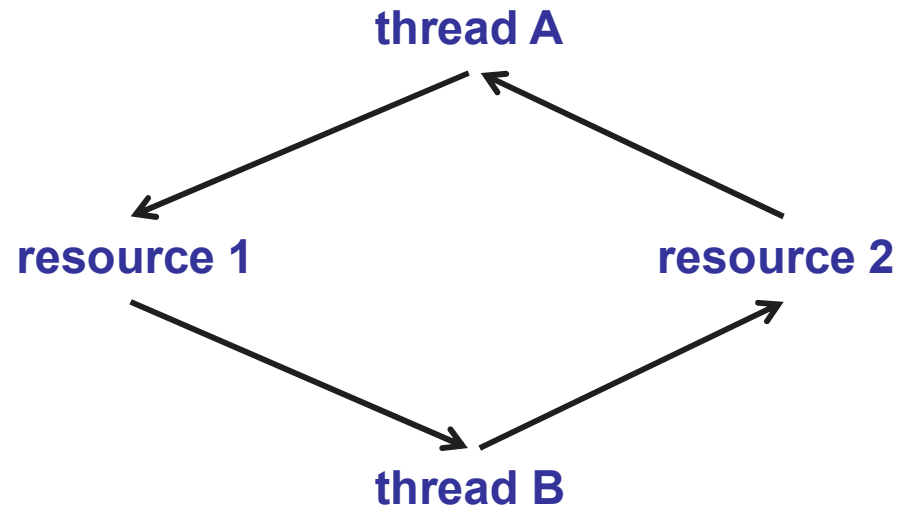
Deadlock

- So far, we have made programs correct by **constraining schedules**
 - Allow only correct interleavings
- But, also possible to **over-constrain** schedules
 - A must happen before B
 - B must happen before A
 - Result is called **deadlock**
- Two types of correctness
 - **Safety**: all actions that occur must be correct
 - **Liveness**: actions must keep occurring. Deadlock violates liveness.

Deadlock

- Resources
 - Things that a thread needs and may need to wait for
 - Examples: locks, disk space, memory, CPU
- Deadlock: a cyclical waiting for resources

Wait-for graph



- Cycle represents a deadlock

Traffic example



Lock example

Thread A

x.lock

y.lock

...

y.unlock

x.unlock

Thread B

y.lock

x.lock

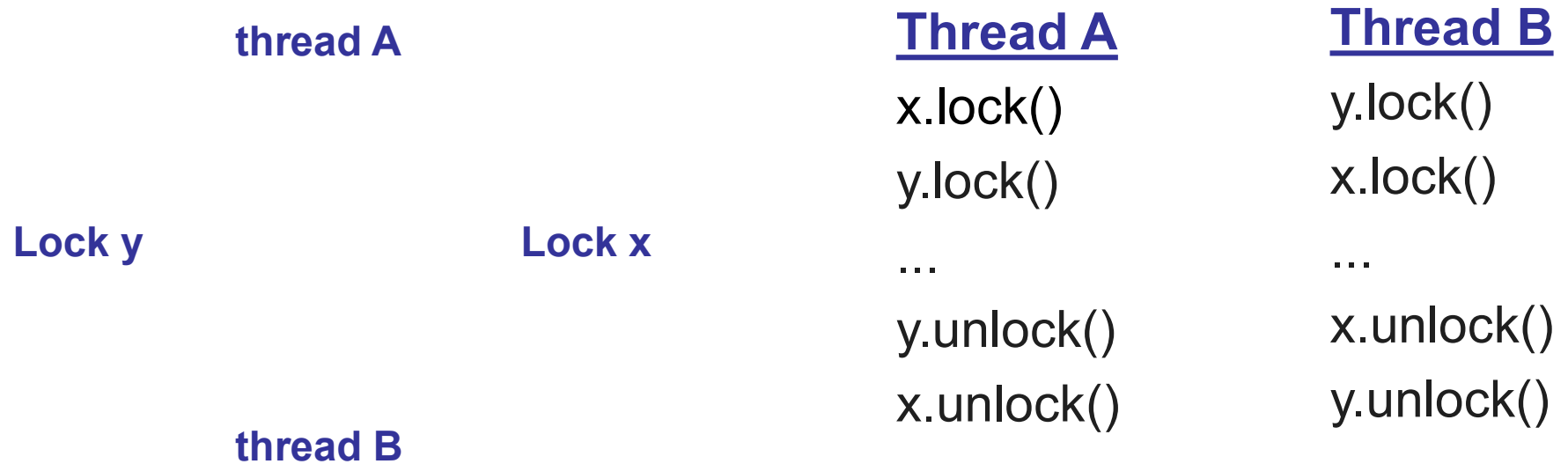
...

x.unlock

y.unlock

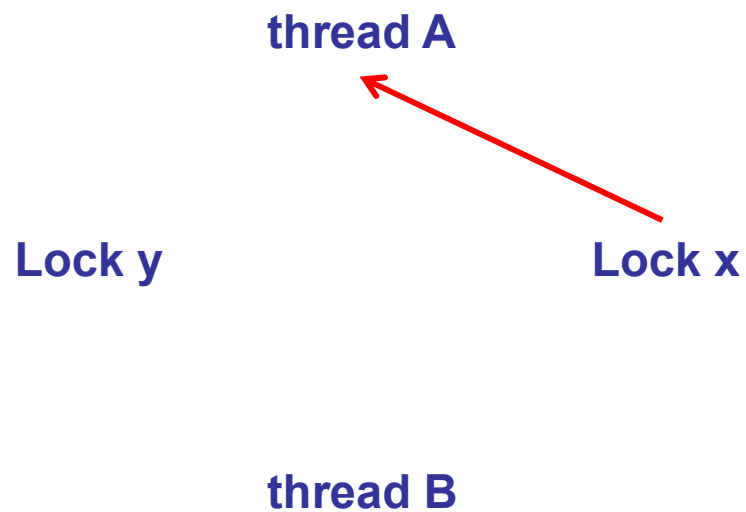
- Can deadlock occur?
- Will deadlock always occur?

Wait-for graph



Cycle represents a deadlock

Wait-for graph



Thread A

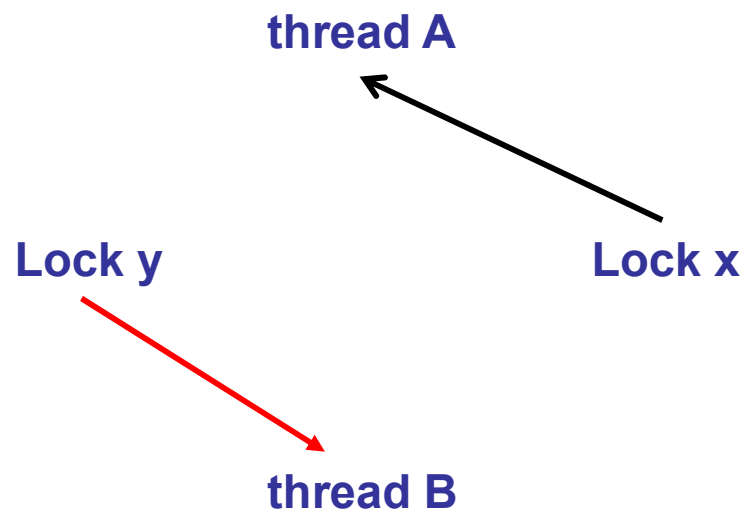
x.lock()
y.lock()
...
y.unlock()
x.unlock()

Thread B

y.lock()
x.lock()
...
x.unlock()
y.unlock()

Cycle represents a deadlock

Wait-for graph



Thread A

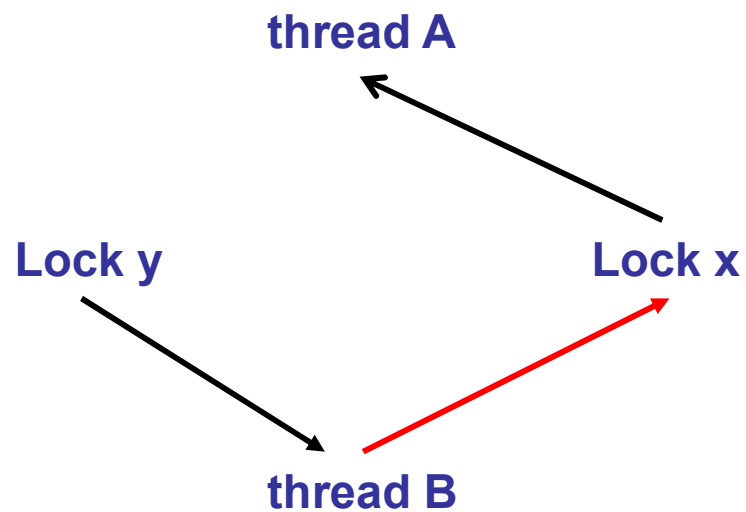
```
x.lock()  
y.lock()  
...  
y.unlock()  
x.unlock()
```

Thread B

```
y.lock()  
x.lock()  
...  
x.unlock()  
y.unlock()
```

Cycle represents a deadlock

Wait-for graph



Thread A

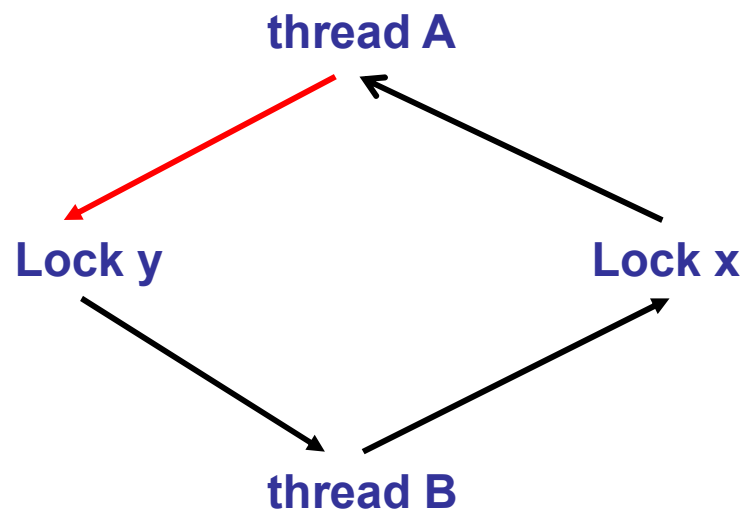
x.lock()
y.lock()
...
y.unlock()
x.unlock()

Thread B

y.lock()
x.lock()
...
x.unlock()
y.unlock()

Cycle represents a deadlock

Wait-for graph



Thread A

x.lock()
y.lock()
...
y.unlock()
x.unlock()

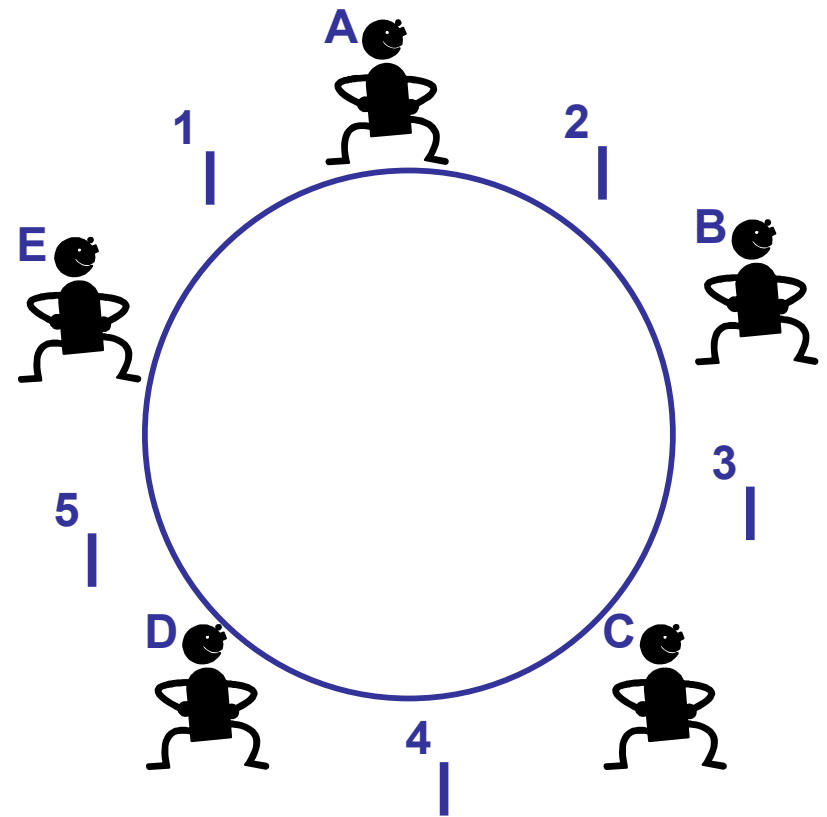
Thread B

y.lock()
x.lock()
...
x.unlock()
y.unlock()

Cycle represents a deadlock

Dining philosophers

- 5 philosophers sit at round table
- 1 chopstick between each pair of philosophers
- Each philosopher needs 2 chopsticks to eat



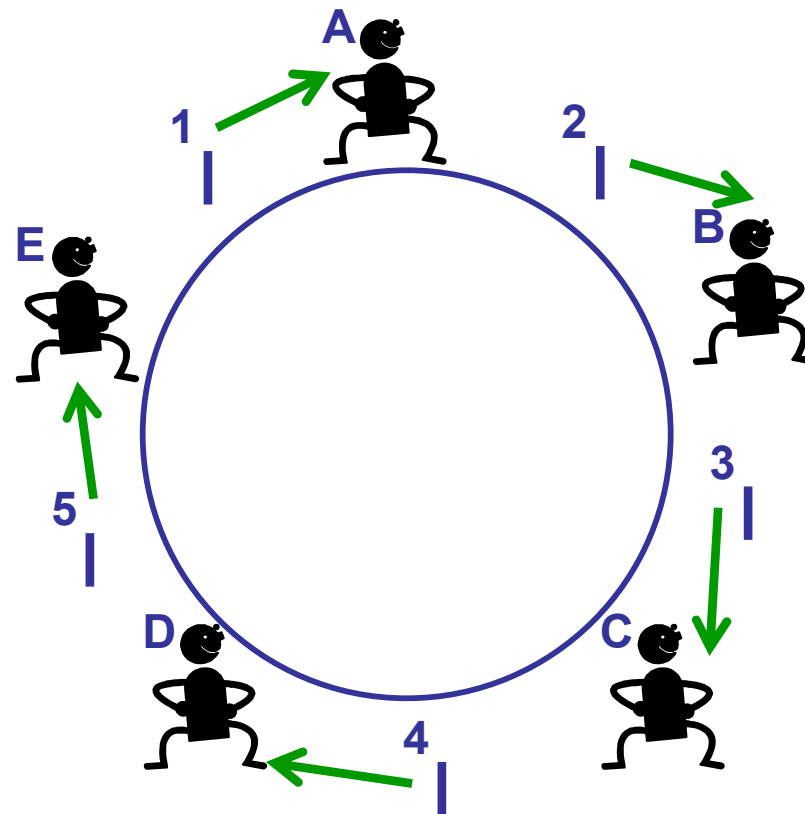
Dining philosophers

Algorithm for philosopher:

1. wait for chopstick on right to be free
2. pick up chopstick on right
3. wait for chopstick on left to be free
4. pick up chopstick on left
5. eat
6. put both chopsticks down

- Can this deadlock?

Dining philosophers



Generic example of multi-threaded program

phase 1

```
while (!done) {  
    acquire some resource  
    work  
}
```

phase 2

```
release all resources
```

What to do about deadlocks

- Ignore
 - Typical OS strategy for application deadlocks
 - Deadlocked threads consume no CPU time
- Detect and fix
 - Detect: look for cycle in wait-for graph
 - Fix #1
 - » Kill one of the threads involved in the deadlock
 - » What about broken invariants?
 - Fix #2
 - » Roll back actions of one of the threads, then try again
 - » This fixes broken invariants
 - » Is it always possible to roll back a thread?

What to do about deadlocks

- Ignore
- Detect and fix
- Prevent
 - Think about what conditions are necessary for deadlock to occur